

Projeto Grafos - Relatorio Tecnico e Manual de Uso

Este relatorio concentra instrucoes de uso, detalhes tecnicos e resultados das duas etapas do projeto: Parte 1 (Grafo dos bairros do Recife) e Parte 2 (Dataset de voos dos EUA).

Indice Geral

1. [Parte 1 - Grafo dos Bairros do Recife](#parte-1-grafo-dos-bairros-do-recife)
 - [Manual de Uso](#manual-de-uso)
 - [Documentacao Tecnica](#documentacao-tecnica)
 - [Algoritmos Implementados](#algoritmos-implementados)
 - [Arquivos de Entrada e Saida](#arquivos-de-entrada-e-saida)
2. [Parte 2 - Dataset de Voos](#parte-2-dataset-de-voos)
 - [Manual de Uso Parte 2](#manual-de-uso-parte-2)
 - [Arquitetura eCodigo](#arquitetura-e-codigo-parte-2)
 - [Algoritmos e Metricas](#algoritmos-e-metricas-parte-2)
 - [Dataset e Saidas](#dataset-e-saidas-parte-2)

Parte 1: Grafo dos Bairros do Recife

Manual de Uso

Requisitos do Sistema

- Python 3.11 ou superior
- Bibliotecas Python (instalar via pip install -r requirements.txt):
 - pandas
 - matplotlib
 - pyvis

Instalacao

1. Clone o repositório:
[trecho de código - veja RELATORIO.md]
2. Instale as dependências:
[trecho de código - veja RELATORIO.md]

Execucao

Opcao 1: Gerar Todos os Arquivos da Parte 1

Para gerar todos os arquivos de saída e iniciar o servidor HTTP para visualização:

[trecho de código - veja RELATÓRIO.md]

Este comando:

- Gera todas as métricas (globais, microrregiões, ego-subrede)
- Calcula graus e rankings
- Calcula distâncias entre endereços
- Gera todas as visualizações (PNG e HTML)
- Inicia servidor HTTP na porta 8000

Acesse os arquivos HTML em:

- http://localhost:8000/grafico_interativo.html
- http://localhost:8000/arvore_percurso.html
- http://localhost:8000/arvore_bfs_boa_vista.html

Opção 2: Executar Algoritmo Específico

BFS (Busca em Largura):

[trecho de código - veja RELATÓRIO.md]

DFS (Busca em Profundidade):

[trecho de código - veja RELATÓRIO.md]

Dijkstra (Caminho Mínimo):

[trecho de código - veja RELATÓRIO.md]

O parâmetro `--target "Boa Viagem"` produz relatórios exibindo Boa Viagem (Setúbal).

Bellman-Ford:

[trecho de código - veja RELATÓRIO.md]

Estrutura de Diretórios

[trecho de código - veja RELATÓRIO.md]

Documentação Técnica

Estrutura do Código

1. `graphs/graph.py` - Classe `Graph`

A classe `Graph` implementa uma estrutura de grafo usando lista de adjacências.

Atributos:

- adj: Dicionario onde cada chave e um vertice e o valor e uma lista de tuplas (vizinho, peso)
- directed: Boolean indicando se o grafo e direcionado (False para Parte 1)

Metodos principais:

- add_vertex(v): Adiciona um vertice ao grafo
- add_edge(u, v, weight, directed): Adiciona aresta entre u e v com peso
- neighbors(v): Retorna lista de vizinhos de v com seus pesos
- vertices(): Retorna lista de todos os vertices
- edges(): Retorna lista de todas as arestas

Caracteristicas:

- Para grafos nao-direcionados (Parte 1), cada aresta e adicionada bidirecionalmente
- Suporta pesos nas arestas
- Implementacao eficiente para grafos esparsos

2. graphs/io.py - Leitura de Dados

Funcoes principais:

- load_nodes(graph, path): Carrega nos do CSV bairros_unique.csv
- load_edges(graph, path): Carrega arestas do CSV adjacencias_bairros.csv
- build_graph(): Constroi o grafo completo dos bairros do Recife

Normalizacao:

- Normaliza nomes de bairros (capitalizacao, remocao de espacos)
- Usa "Boa Viagem" como no canonico para Setubal e exibe "Boa Viagem (Setubal)" nas saidas

3. graphs/algorithms.py - Implementacao dos Algoritmos

Todos os algoritmos foram implementados manualmente, sem usar bibliotecas externas.

4. solve.py - Orquestracao

Contem funcoes que orquestram as analises:

- run_metricas_globais(): Calcula metricas do grafo completo
- run_metricas_microrregioes(): Calcula metricas por microrregiao
- run_ego_bairros(): Calcula ego-subrede para cada bairro
- run_graus(): Calcula grau de cada bairro
- run_dist_enderecos(): Calcula distancias entre enderecos usando Dijkstra
- run_percurso_nova_descoberta_setubal(): Calcula caminho especifico
- run_arvore_percurso(): Gera visualizacoes da arvore do percurso
- run_grafo_interativo(): Gera grafo interativo HTML
- run_arvore_bfs(): Gera arvore BFS com camadas

5. viz.py - Visualizacoes

Funcoes para gerar visualizacoes:

- gerar_distribuicao_graus(): Histograma de distribuicao de graus

- gerar_top10_grau(): Grafico de barras dos 10 bairros com maior grau
- gerar_densidade_ego_microrregiao(): Grafico de densidade por microrregiao
- gerar_arvore_percurso_png(): Arvore do percurso em PNG (matplotlib)
- gerar_arvore_percurso_html(): Arvore do percurso em HTML (pyvis)
- gerar_grafo_interativo(): Grafo completo interativo com busca
- gerar_arvore_bfs(): Arvore BFS com camadas (PNG e HTML)

Algoritmos Implementados

1. BFS (Breadth-First Search) - Busca em Largura

Localizacao: graphs/algorithms.py, funcao bfs()

Descricao:

Algoritmo de busca em largura que explora o grafo nivel por nivel, comecando de um vertice origem.

Implementacao:

[trecho de codigo - veja RELATORIO.md]

Caracteristicas:

- Usa fila (lista Python) para manter ordem de visitacao
- Calcula camadas (niveis) de cada vertice
- Detecta ciclos em grafos nao-direcionados
- Complexidade: $O(V + E)$ onde V = vertices, E = arestas

Retorno:

- ordem: Lista de vertices na ordem de visitacao
- camadas: Dicionario vertice nivel
- ciclos: Lista de arestas que formam ciclos

2. DFS (Depth-First Search) - Busca em Profundidade

Localizacao: graphs/algorithms.py, funcao dfs()

Descricao:

Algoritmo de busca em profundidade que explora o grafo seguindo um caminho ate o fim antes de retroceder.

Implementacao:

[trecho de codigo - veja RELATORIO.md]

Caracteristicas:

- Implementacao recursiva
- Classifica arestas em "tree" (arestas da arvore) e "back" (arestas de retorno/ciclo)
- Detecta ciclos em grafos nao-direcionados
- Complexidade: $O(V + E)$

Retorno:

- ordem: Lista de vertices na ordem de visitacao
- ciclos: Lista de arestas que formam ciclos
- arestas_class: Lista de arestas classificadas

3. Dijkstra - Caminho Minimo

Localizacao: graphs/algorithms.py, funcao dijkstra()

Descricao:

Algoritmo para encontrar o caminho de menor custo entre dois vertices em um grafo com pesos nao-negativos.

Implementacao:

[trecho de codigo - veja RELATORIO.md]

Caracteristicas:

- Implementacao sem heapq: Usa busca linear para encontrar o minimo (conforme requisito)
- Funciona apenas com pesos nao-negativos
- Para quando encontra o destino (otimizacao)
- Complexidade: $O(V^2 + E)$ com busca linear ($O(V \log V + E)$ com heap)

Retorno:

- Tupla (distancia, caminho) onde:
 - distancia: Custo total do caminho minimo
 - caminho: Lista de vertices do caminho

4. Bellman-Ford - Caminho Minimo com Deteccao de Ciclos Negativos

Localizacao: graphs/algorithms.py, funcao bellman_ford()

Descricao:

Algoritmo para encontrar caminhos minimos que funciona com pesos negativos e detecta ciclos negativos.

Implementacao:

[trecho de codigo - veja RELATORIO.md]

Caracteristicas:

- Funciona com pesos negativos
- Detecta ciclos negativos
- Relaxa arestas $(|V|-1)$ vezes
- Complexidade: $O(V \cdot E)$

Retorno:

- Tupla (dist, pai, ciclo_negativo) onde:
 - dist: Dicionario de distancias minimas
 - pai: Dicionario de predecessores (para reconstruir caminhos)
 - ciclo_negativo: Boolean indicando se ha ciclo negativo

Arquivos de Entrada e Saida

Arquivos de Entrada (data/)

bairros_recife.csv

Dataset original com colunas "1.1" a "6.3" contendo bairros agrupados por microrregioes.

bairros_unique.csv

Bairros normalizados e unicos. Formato:

[trecho de codigo - veja RELATORIO.md]

adjacencias_bairros.csv

Arestas do grafo. Formato:

[trecho de codigo - veja RELATORIO.md]

Regua de Pesos:

- 1 = Rua (vias locais)
- 2 = Ponte (travessias, viadutos)
- 3 = Avenida (vias arteriais principais)
- 4 = Rodovia (vias expressas)
- 5 = Estrada (estradas principais)

enderecos.csv

Pares de enderecos para analise. Formato:

[trecho de codigo - veja RELATORIO.md]

Arquivos de Saida (out/)

recife_global.json

Metricas globais do grafo:

[trecho de codigo - veja RELATORIO.md]

microrregioes.json

Metricas por microrregiao:

[trecho de codigo - veja RELATORIO.md]

ego_bairro.csv

Ego-subrede por bairro:

[trecho de código - veja RELATORIO.md]

graus.csv

Grau de cada bairro:

[trecho de código - veja RELATORIO.md]

distancias_enderecos.csv

Distancias e caminhos entre enderecos:

[trecho de código - veja RELATORIO.md]

percurso_nova_descoberta_setubal.json

Caminho específico Nova Descoberta Boa Viagem (Setubal):

[trecho de código - veja RELATORIO.md]

Visualizacoes

PNG (matplotlib):

- distribuicao_graus.png: Histograma da distribuicao de graus
- top10_grau.png: Top 10 bairros por grau
- densidade_ego_microrregiao.png: Densidade media por microrregiao
- arvore_percurso.png: Arvore do percurso estatica
- arvore_bfs_boa_vista.png: Arvore BFS com camadas

HTML (pyvis):

- grafo_interativo.html: Grafo completo com:
 - Tooltip por bairro (grau, microrregiao, densidade_ego)
 - Campo de busca por bairro
 - Caminho "Nova Descoberta Boa Viagem (Setubal)" destacado em vermelho
- arvore_percurso.html: Arvore do percurso interativa
- arvore_bfs_boa_vista.html: Arvore BFS interativa

notas_analiticas.md

Documento explicando o que cada visualizacao revela sobre o grafo.

Parte 2: Dataset de Voos

Manual de Uso Parte 2

1. Preparacao do ambiente

```
`bash
cd parte2
source ../.venv/bin/activate # se ainda nao estiver ativo
`
```

2. Execucao completa (relatorio, metricas e graficos):

```
`bash
python3 -m src.cli --out ./out/
`
```

O comando carrega data/voos.csv, executa todas as analises, gera out/parte2_report.json e salva as visualizacoes PNG.

3. Algoritmos especificos:

- BFS: `python3 -m src.cli --alg BFS --source EWR --out ./out/`
- DFS: `python3 -m src.cli --alg DFS --source EWR --out ./out/`
- Dijkstra: `python3 -m src.cli --alg DIJKSTRA --source EWR --target LAX --out ./out/`
- Bellman-Ford: `python3 -m src.cli --alg BELLMAN_FORD --source EWR --out ./out/`

4. Testes automatizados:

```
`bash
pytest tests/
`
```

Executa casos unitarios cobrindo BFS, DFS, Dijkstra e Bellman-Ford em cenarios direcionados.

Arquitetura eCodigo Parte 2

- src/cli.py: Interface de linha de comando que valida argumentos, configura diretorios e delega execucoes pontuais ou a analise completa.
- src/algorithms2.py:
 - DiGraph: especializacao dirigida da classe Graph da Parte 1, reutilizando toda a infraestrutura.
 - carregar_grafo_voos(): le data/voos.csv (colunas ORIGIN, DEST, DISTANCE) e monta o grafo dirigido ponderado.
 - rodar_analise_parte2(): orquestra BFS/DFS para EWR, JFK, LGA, roda Dijkstra para 5 pares, Bellman-Ford para multiplas fontes, mede tempo/memoria e gera parte2_report.json.
 - Funcoes auxiliares para graficos (gerar_distribuicao_grau_voos, gerar_top_hubs_voos, gerar_histograma_distancias_voos, gerar_disp_grau_in_out_voos, gerar_top_rotas_distantes).
- tests/: suite pytest que garante a corretude dos algoritmos com grafos de voo reduzidos e cenarios sinteticos com ciclo negativo.

Algoritmos e Metricas Parte 2

- BFS/DFS: reutilizam as implementacoes da Parte 1, agora explorando grafo direcionado; o relatorio registra ordem de visitacao, camadas e ciclos detectados por fonte.
- Dijkstra: calcula caminhos minimos entre pares de hubs obrigatorios (EWR, JFK, LGA, SFO, LAX, ATL, MIA), registrando custo, caminho reconstruido, tempo e memoria.
- Bellman-Ford: roda em duas configuracoes (grafo real e cenarios artificiais com ciclo negativo) para validar estabilidade com pesos positivos e negativos.

- Metricas globais: numero de vertices/arestas, pares direcionados unicos, estatisticas de grau (minimo, maximo, media, mediana), top hubs e distribuicao de distancias.
- Monitoramento de desempenho: `rodar_analise_parte2()` usa `tracemalloc` e `time.perf_counter()` para anexar consumo de memoria (KB) e tempo (s) por tarefa, facilitando comparacoes entre algoritmos.

Dataset e Saidas Parte 2

- Dataset `data/voos.csv`:
 - Colunas principais: `ORIGIN`, `DEST`, `DISTANCE`.
 - Mais de 300 rotas direcionadas entre aeroportos norte-americanos, com pesos em milhas.
 - Durante a carga, pares invalidos sao ignorados e distancias sao convertidas para float.
- Arquivos gerados em `parte2/out/`:
 - `parte2_report.json`: resumo completo das metricas, execucoes e resultados dos algoritmos.
 - `grau_distribuicao.png`: histograma do grau total (entrada+saida).
 - `top_hubs_grau.png`: ranking de hubs com maior conectividade.
 - `histograma_distancias.png`: distribuicao de distancias das rotas.
 - `grau_in_out_scatter.png`: correlacao entre graus de entrada e saida (tamanho proporcional ao grau total).
 - `top_rotas_distantes.png`: barras horizontais com as rotas mais longas.

Consideracoes Tecnicas

Complexidade dos Algoritmos

Algoritmo	Complexidade	Observacao
----- ----- -----		
BFS	$O(V + E)$	Otima para grafos esparsos
DFS	$O(V + E)$	Otima para grafos esparsos
Dijkstra	$O(V^2 + E)$	Com busca linear (sem heap)
Bellman-Ford	$O(V \cdot E)$	Funciona com pesos negativos

Estrutura de Dados

- Lista de Adjacencias: Usada para representar o grafo
 - Vantagem: Eficiente para grafos esparsos
 - Espaco: $O(V + E)$
 - Acesso a vizinhos: $O(\text{grau do vertice})$

Normalizacao de Dados

- Nomes de bairros sao normalizados (capitalizacao, remocao de espacos)
- "Setubal" e tratado como "Boa Viagem" no grafo e rotulado como "Boa Viagem (Setubal)" para o usuario
- Acentuacao e preservada

Tratamento de Casos Especiais

- Grafo vazio: Densidade = 0
- Vertice isolado: Densidade = 0
- Caminho inexistente: Dijkstra retorna float('inf')
- Ciclos negativos: Bellman-Ford detecta e retorna flag

Conclusao

Este projeto implementa algoritmos fundamentais de grafos (BFS, DFS, Dijkstra, Bellman-Ford) do zero, sem usar bibliotecas que ja os implementam. As aplicacoes praticas cobrem:

- Parte 1: analise do grafo dos bairros do Recife, com metricas globais/locais, caminhos minimos e visualizacoes interativas/estaticas.
- Parte 2: exploracao de um grafo dirigido de voos, avaliando hubs, rotas criticas, distribuicao de distancias e desempenho dos algoritmos em um dataset maior.

Ambas as etapas possuem documentacao, relatorios e scripts reproduziveis, permitindo replicar resultados e estender o trabalho para novas fontes de dados.

Versao: 1.0

Data: 2025

Python: 3.11+